

Burp Suite - Intruder

Burp Suite's Intruder module is a powerful tool that allows for automated and customisable attacks. It provides the ability to modify specific parts of a request and perform repetitive tests with variations of input data. Intruder is particularly useful for tasks like fuzzing and brute-forcing, where different values need to be tested against a target.

What is Intruder

Intruder can send multiple requests with slightly altered values based on user-defined configurations

brute-forcing login forms by substituting username and password fields with values from a wordlist or performing fuzzing attacks using wordlists to test subdirectories, endpoints, or virtual hosts



There are four sub-tabs within Intruder:

- **Positions:** This tab allows us to select an attack type and configure where we want to insert our payloads in the request template.
- **Payloads:** Here we can select values to insert into the positions defined in the **Position** tab. We have various payload options, such as loading items from a wordlist. The way these payloads are inserted into the template depends on the attack type chosen in the **Positions** tab. The **Payloads** tab also enables us to modify Intruder's behaviour regarding payloads, such as defining pre-processing rules for each payload (e.g., adding a

prefix or suffix, performing match and replace, or skipping payloads based on a defined regex).

- **Resource Pool:** This tab is not particularly useful in the Burp Community Edition. It allows for resource allocation among various automated tasks in Burp Professional. Without access to these automated tasks, this tab is of limited importance.
- **Settings:** This tab allows us to configure attack behaviour. It primarily deals with how Burp handles results and the attack itself. For instance, we can flag requests containing specific text or define Burp's response to redirect (3xx) responses.

The term "fuzzing" refers to the process of testing functionality or existence by applying a set of data to a parameter. For example, fuzzing for endpoints in a web application involves taking each word in a wordlist and appending it to a request URL

(e.g., http://10.130.184.248/WORD_GOES_HERE) to observe the server's response.

Positions

On the right-hand side of the interface, we find the following buttons: `Add §`, `Clear §`, and `Auto §`:

- The `Add §` button allows us to define new positions manually by highlighting them within the request editor and then clicking the button.
- The `Clear §` button removes all defined positions, providing a blank canvas where we can define our own positions.
- The `Auto §` button automatically attempts to identify the most likely positions based on the request. This feature is helpful if we previously cleared the default positions and want them back.

Payloads

Dashboard
Target
Proxy
Intruder
Repeater
Collaborator
Sequencer
Decoder
Comparer
Logger
Organizer
Extensions
Settings
Learn

1 x
3 x
+

Positions
Payloads
Resource pool
Settings

1 Payload sets

You can define one or more payload sets. The number of payload sets depends on the attack type defined in the Positions tab. Various payload types are available for each payload set, and each payload type can be customized in different ways.

Payload set: 1
Payload count: 0

Payload type: Simple list
Request count: 0

Start attack

2 Payload settings [Simple list]

This payload type lets you configure a simple list of strings that are used as payloads.

Paste
Load ...
Remove
Clear
Deduplicate

Add
Enter a new item

Add from list ... [Pro version only]

3 Payload processing

You can define rules to perform various processing tasks on each payload before it is used.

Add
Edit
Remove
Up
Down

Enabled	Rule

4 Payload encoding

This setting can be used to URL-encode selected characters within the final payload, for safe transmission within HTTP requests.

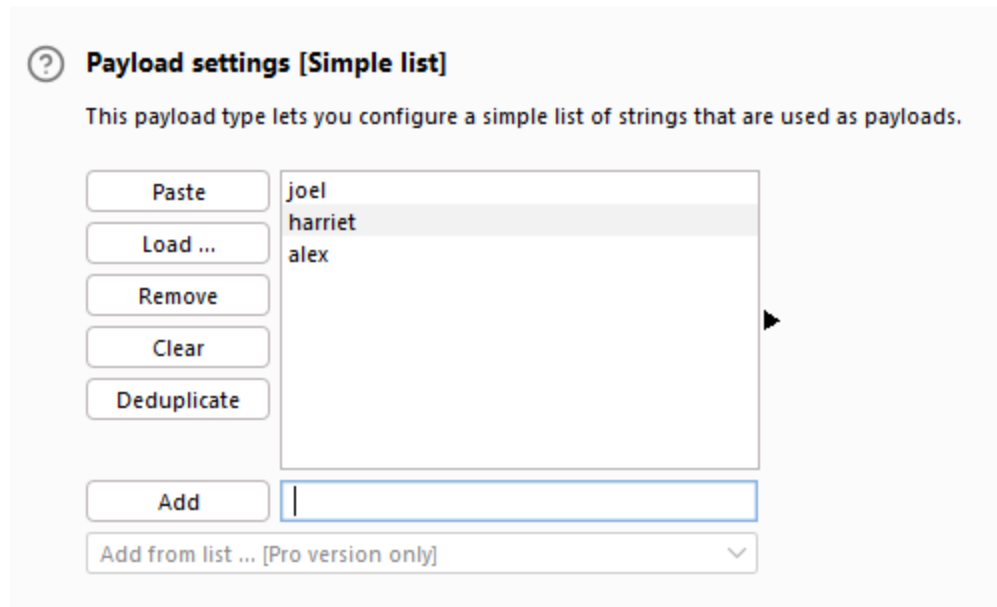
☒ URL-encode these characters: .\^=<>?+&*;"'[]{}

1. Payload Sets:

- This section allows us to choose the position for which we want to configure a payload set and select the type of payload we want to use.
- When using attack types that allow only a single payload set (Sniper or Battering Ram), the "Payload Set" dropdown will have only one option, regardless of the number of defined positions.
- If we use attack types that require multiple payload sets (Pitchfork or Cluster Bomb), there will be one item in the dropdown for each position.
- Note:** When assigning numbers in the "Payload Set" dropdown for multiple positions, follow a top-to-bottom, left-to-right order. For example, with two positions (`username=$pentester$&password=$Exp101ted$`), the first item in the payload set dropdown would refer to the username field, and the second item would refer to the password field.

2. Payload settings:

- This section provides options specific to the selected payload type for the current payload set.
- For example, when using the "Simple list" payload type, we can manually add or remove payloads to/from the set using the **Add** text box, **Paste** lines, or **Load** payloads from a file. The **Remove** button removes the currently selected line, and the **Clear** button clears the entire list. Be cautious with loading huge lists, as it may cause Burp to crash.
- Each payload type will have its own set of options and functionality. Explore the options available to understand the range of possibilities.



3. Payload Processing:

- In this section, we can define rules to be applied to each payload in the set before it is sent to the target.
- For example, we can capitalize every word, skip payloads that match a regex pattern, or apply other transformations or filtering.
- While you may not use this section frequently, it can be highly valuable when specific payload processing is required for your attack.

4. Payload Encoding:

- The section allows us to customize the encoding options for our payloads.
- By default, Burp Suite applies URL encoding to ensure the safe transmission of payloads. However, there may be cases where we want to adjust the encoding behavior.
- We can override the default URL encoding options by modifying the list of characters to be encoded or unchecking the "URL-encode these characters" checkbox.

Introduction to Attack Types

The **Positions** tab of Burp Suite Intruder has a dropdown menu for selecting the attack type. Intruder offers four attack types, each serving a specific purpose. Let's explore each of them:

1. **Sniper**: The Sniper attack type is the default and most commonly used option. It cycles through the payloads, inserting one payload at a time into each position defined in the request. Sniper attacks iterate through all the payloads in a linear fashion, allowing for precise and focused testing.
2. **Battering ram**: The Battering ram attack type differs from Sniper in that it sends all payloads simultaneously, each payload inserted into its respective position. This attack type is useful when testing for race conditions or when payloads need to be sent concurrently.
3. **Pitchfork**: The Pitchfork attack type enables the simultaneous testing of multiple positions with different payloads. It allows the tester to define multiple payload sets, each associated with a specific position in the request. Pitchfork attacks are effective when there are distinct parameters that need separate testing.
4. **Cluster bomb**: The Cluster bomb attack type combines the Sniper and Pitchfork approaches. It performs a Sniper-like attack on each position but simultaneously tests all payloads from each set. This attack type is useful when multiple positions have different payloads, and we want to test them all together.

Each attack type has its advantages and is suitable for different testing scenarios.

Understanding their differences helps us select the appropriate attack type based on the testing objectives.

Sniper

default and most commonly used attack type in Burp Suite Intruder. It is particularly effective for single-position attacks, such as password brute-force or fuzzing for API endpoints

Sniper attack, we provide a set of payloads, which can be a wordlist or a range of numbers, and Intruder inserts each payload into each defined position in the request.

```
Example Positions

POST /support/login/ HTTP/1.1
Host: 10.129.134.189
User-Agent: Mozilla/5.0 (X11; Ubuntu; Linux x86_64; rv:80.0) Gecko/20100101 Firefox
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/webp,*/*;q=0.8
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate
Content-Type: application/x-www-form-urlencoded
Content-Length: 37
Origin: http://10.129.134.189
Connection: close
Referer: http://10.129.134.189/support/login/
Upgrade-Insecure-Requests: 1

username=$pentester$&password=$Exp101ted$
```

Assuming we have a wordlist with three words: burp , suite , and intruder , Intruder would generate six requests:

Request Number	Request Body
1	username=burp&password=Exp101ted
2	username=suite&password=Exp101ted
3	username=intruder&password=Exp101ted
4	username=pentester&password=burp
5	username=pentester&password=suite
6	username=pentester&password=intruder

The total number of requests made by Intruder Sniper can be calculated as `requests = numberOfWords * numberOfPositions`

Battering Ram

The **Battering ram** attack type in Burp Suite Intruder differs from Sniper in that it places the same payload in every position simultaneously, rather than substituting each payload into each position in turn.

```
Example Positions

POST /support/login/ HTTP/1.1
Host: 10.129.134.189
User-Agent: Mozilla/5.0 (X11; Ubuntu; Linux x86_64; rv:80.0) Gecko/20100101 Firefox/80.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/webp,*/*;q=0.8
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate
Content-Type: application/x-www-form-urlencoded
Content-Length: 37
Origin: http://10.129.134.189
Connection: close
Referer: http://10.129.134.189/support/login/
Upgrade-Insecure-Requests: 1

username=$pentester$&password=$Exploited$
```

Using the Battering Ram attack type with the same wordlist from before (burp , suite , and intruder), Intruder would generate three requests:

Request Number	Request Body
1	username=burp&password=burp
2	username=suite&password=suite
3	username=intruder&password=intruder

Pitchfork

The **Pitchfork** attack type in Burp Suite Intruder is similar to having multiple Sniper attacks running simultaneously. While Sniper uses one payload set to test all positions simultaneously, Pitchfork utilises one payload set per position (up to a maximum of 20) and iterates through them all simultaneously.

To better understand Pitchfork, let us revisit our brute-force example, but this time with two wordlists:

1. The first wordlist contains usernames: joel , harriet , and alex .
2. The second wordlist contains passwords: J03l , Emma1815 , and Sk1ll .

Request Number	Request Body
1	username=joel&password=J03l
2	username=harriet&password=Emma1815
3	username=alex&password=Sk1ll

The Pitchfork attack type is especially useful when conducting credential-stuffing attacks or when multiple positions require separate payload sets. It allows for simultaneous testing of multiple positions with different payloads.

Cluster Bomb

The Pitchfork attack type is especially useful when conducting credential-stuffing attacks or when multiple positions require separate payload sets. It allows for simultaneous testing of multiple positions with different payloads.

Request Number	Request Body
1	<code>username=joel&password=J031</code>
2	<code>username=harriet&password=J031</code>
3	<code>username=alex&password=J031</code>
4	<code>username=joel&password=Emma1815</code>
5	<code>username=harriet&password=Emma1815</code>
6	<code>username=alex&password=Emma1815</code>
7	<code>username=joel&password=Sk111</code>
8	<code>username=harriet&password=Sk111</code>
9	<code>username=alex&password=Sk111</code>

Cluster bomb attack type iterates through every combination of the provided payload sets. It tests every possibility by substituting each value from each payload set into the corresponding position in the request.

The Cluster bomb attack type is particularly useful for credential brute-forcing scenarios where the mapping between usernames and passwords is unknown.

Practical Example

here is a login form

```
Support Login Form Source Code

---
<form method="POST">
  <div class="form-floating mb-3">
    <input class="form-control" type="text" name=username placeholder="Usernam
    <label for="username">Username</label>
  </div>
  <div class="form-floating mb-3">
    <input class="form-control" type="password" name=password placeholder="Pas
    <label for="password">Password</label>
  </div>
  <div class="d-grid"><button class="btn btn-primary btn-lg" type="submit">Login!
</form>
---
```

It has no protective measures meaning there is multiple ways to exploit this form one being a cluster bomb attack to brute force the credentials

I captured the login request in burpsuite and sent it to the intruder I then selected the type of attack and then entered the username and password parameter with the special character on

DashboardTargetProxyIntruderRepeaterCollaboratorSequencerDecoderComparerLoggerOrganizerExtensionsLearn

1 x2 x+

?

Pitchfork attack

Start attack

Target

http://10.130.174.156

Update Host header to match target

Add \$

Clear \$

Auto \$

1 POST /support/login/ HTTP/1.1

2 Host: 10.130.174.156

3 User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:147.0) Gecko/20100101 Firefox/147.0

4 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8

5 Accept-Language: en-GB,en;q=0.9

6 Accept-Encoding: gzip, deflate, br

7 Content-Type: application/x-www-form-urlencoded

8 Content-Length: 29

9 Origin: http://10.130.174.156

10 Connection: keep-alive

11 Referer: http://10.130.174.156/support/login/

12 Upgrade-Insecure-Requests: 1

13 Priority: u=0, i

14

15 username=\$username\$&password=\$password\$

Payloads

Payload position: 1

Payload type: Simple list

Payload count: 0

Request count: 0

Payload configuration

This payload type lets you configure a simple list of strings that are used as payloads.

Paste

Load...

Remove

Clear

Deduplicate

Add

Enter a new item

Add from list... [Pro version only]

Payload processing

You can define rules to perform various processing tasks on each payload before it is used.

Add

Edit

Remove

Up

Down

Enabled

Rule

Payload encoding

I then went into the payloads and loaded the usernames into payload position 1 and the passwords into payload position 2

Payloads



Payload position: 1

Payload type: Simple list

Payload count: 100

Request count: 0

Payload configuration

This payload type lets you configure a simple list of strings that are used as payloads

Paste

Load...

Remove

Clear

Deduplicate

l.madden

j.poole

i.hopkins

l.mayo

m.roberts

m.ratliff

j.morrison

r.carroll

t.nichols

d.decker

l.pena

Add

Enter a new item

Add from list... [Pro version only]

Payload processing

Payload position: 2

Payload type: Simple list

Payload count: 100

Request count: 100

Payload configuration

This payload type lets you configure a simple list of strings that are used as payloads.

Paste

Load...

Remove

Clear

Deduplicate

bambam

UnitedKingdom123

1q2q3q

valencia

258852

10031991

wolverine

12111991

asdfghj

02011976

0000000

Add

Enter a new item

Add from list... [Pro version only]

Payload processing

I then hit the attack button to begin the attack which opened this window:

3. Intruder attack of http://10.130.174.156

Attack Save

Results Positions

Intruder attack results filter: Showing all items

Request	Payload 1	Payload 2	Status code	Response r...	Error	Timeout	Length	Comment
0			302	4			680	
1	l.madden	bambam	302	4			680	
2	j.poole	UnitedKingdom123	302	4			680	
3	i.hopkins	1q2q3q	302	4			680	
4	l.mayo	valencia	302	4			680	
5	m.roberts	258852	302	4			680	
6	m.ratliff	10031991	302	5			680	
7	j.morrison	wolverine	302	4			680	
8	r.carroll	12111991	302	4			680	
9	t.nichols	asdfghj	302	4			680	
10	d.decker	02011976	302	4			680	

Since the response status codes are not differentiating successful and unsuccessful attempts (all are 302 redirects), we need to use the response length to distinguish them.

To confirm the successful login attempt, use the credentials from the request with the shorter response length to log in.

the credentials with the shortest response length was:

3. Intruder attack of http://10.130.174.156

Attack Save

Results Positions

Intruder attack results filter: Showing all items

Request	Payload 1	Payload 2	Status code	Response...	Error	Timeout	Length	Comment
50	m.rivera	letmein1	302	318			599	
29	r.gallagher	nicole1	302	9			680	
22	a.mcmahon	fktyrf	302	6			680	
74	j.stout	beemer	302	6			680	
6	m.ratliff	10031991	302	5			680	
67	j.bird	blades	302	5			680	
73	c.olsen	twilight	302	5			680	
77	z.newton	88888	302	5			680	
82	c.farley	11051988	302	5			680	
85	j.wolfe	elvira	302	5			680	
88	d.nordwin	bmw325	302	5			680	

Request Response

Pretty Raw Hex

```
1 POST /support/login/ HTTP/1.1
2 Host: 10.130.174.156
3 User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:147.0) Gecko/20100101 Firefox/147.0
4 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
5 Accept-Language: en-GB,en;q=0.9
6 Accept-Encoding: gzip, deflate, br
7 Content-Type: application/x-www-form-urlencoded
8 Content-Length: 37
9 Origin: http://10.130.174.156
10 Connection: keep-alive
11 Referer: http://10.130.174.156/support/login/
12 Upgrade-Insecure-Requests: 1
13 Priority: u=0, i
14
15 username=m%20rivera&password=letmein1
```

Practical Challenge

<http://10.130.174.156/support/ticket/NUMBER>

1. **Access Control:** The endpoint may be properly configured to restrict access only to tickets assigned to our current user. In this case, we can only view tickets associated with our account.
2. **IDOR Vulnerability:** Alternatively, the endpoint may lack appropriate access controls, leading to a vulnerability known as **Insecure Direct Object References (IDOR)**. If this is the case, we could potentially exploit the system and read all existing tickets, regardless of the assigned user.

As we are using numbers I can set the payload type to numbers

Dashboard

Target

Proxy

Intruder

Repeater

Collaborator

Sequencer

Decoder

Comparer

Logger

Organizer

Extensions

Learn

1 x

2 x

3 x

+

? Sniper attack

Start attack

Target http://10.130.174.156

Update Host header to match target

Add §

Clear §

Auto §

1 GET /support/ticket/\$§ HTTP/1.1

2 Host: 10.130.174.156

3 User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:147.0) Gecko/20100101 Firefox/147.0

4 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8

5 Accept-Language: en-GB,en;q=0.9

6 Accept-Encoding: gzip, deflate, br

7 Connection: keep-alive

8 Referer: http://10.130.174.156/support/

9 Cookie: session=eyJTdXBwb3J0QXV0aCI6IlRydwUiLCJTdXBwb3J0SUQiOiN9LmFVGgq.SlVgtgBNx7cCuNWwEUTxUF2_G4

10 Upgrade-Insecure-Requests: 1

11 Priority: u=0, i

12

Payloads

All payload positions

Numbers

100

100

Payload configuration

This payload type generates numeric payloads within a given range and in a specified format.

Number range

Type:

SequentialRandom

From:

1

To:

100

Step:

1

How many:

Number format

Base:

DecimalHex

Min integer digits:

0

Max integer digits:

3

Min fraction digits:

0

Max fraction digits:

0

Examples

1

321

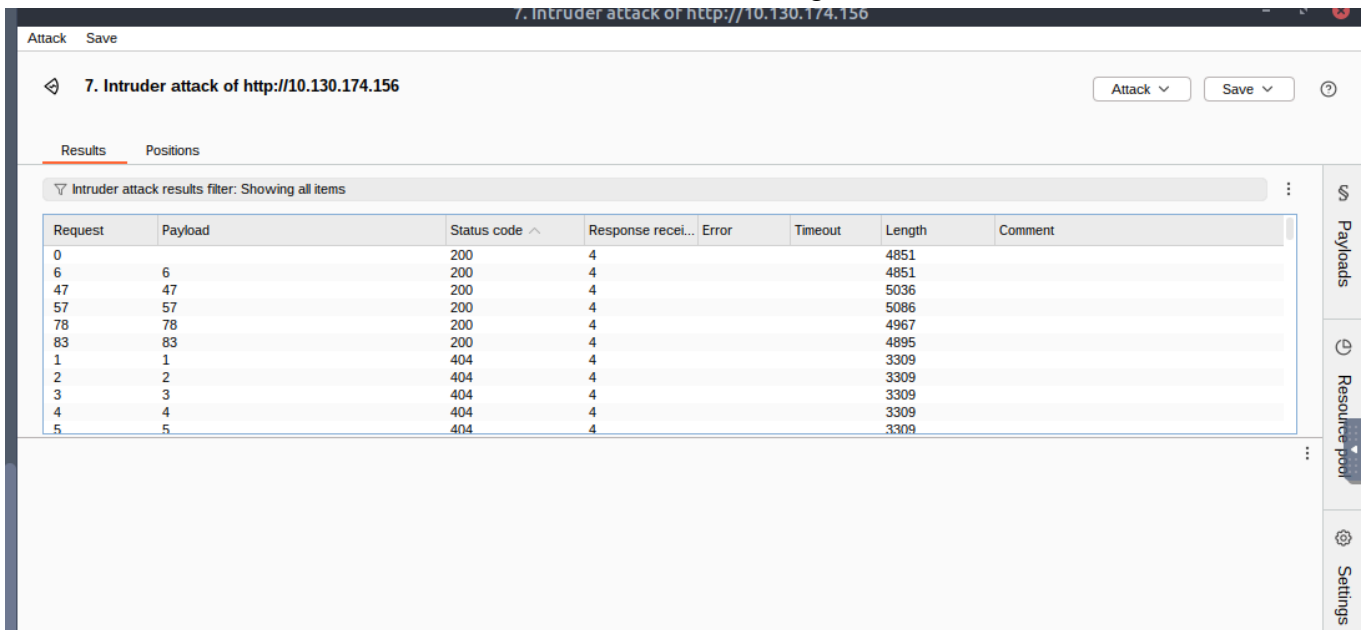
Payload processing

You can define rules to perform various processing tasks on each payload before it is used.

and i have set it from 1 to 100

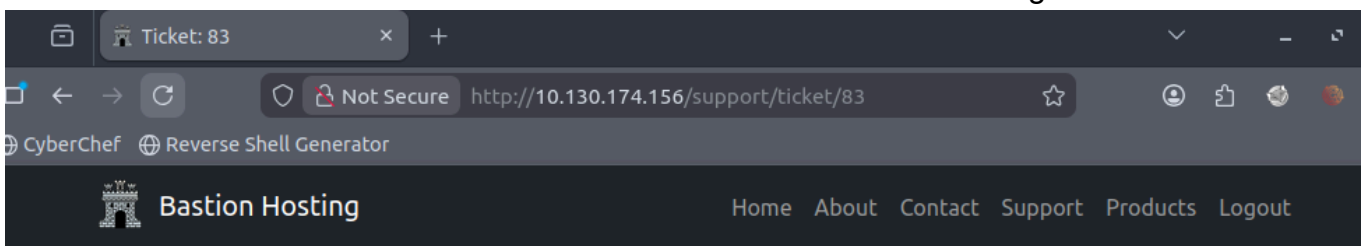
the position was set at the ticket number in the request

here are the ticket numbers with status code 200 meaning these are valid tickets:



Request	Payload	Status code	Response received	Error	Timeout	Length	Comment
0		200	4			4851	
6	6	200	4			4851	
47	47	200	4			5036	
57	57	200	4			5086	
78	78	200	4			4967	
83	83	200	4			4895	
1	1	404	4			3309	
2	2	404	4			3309	
3	3	404	4			3309	
4	4	404	4			3309	
5	5	404	4			3309	

on the website itself I then tried these ticket numbers until i found the flag



Ticket 83

Email
flag@bastionhosting.thm

Query
THM{MTMxNTg5NTUzMWM0OWRlYzUzMDVjMzJl}

Ticket Resolved: ☒

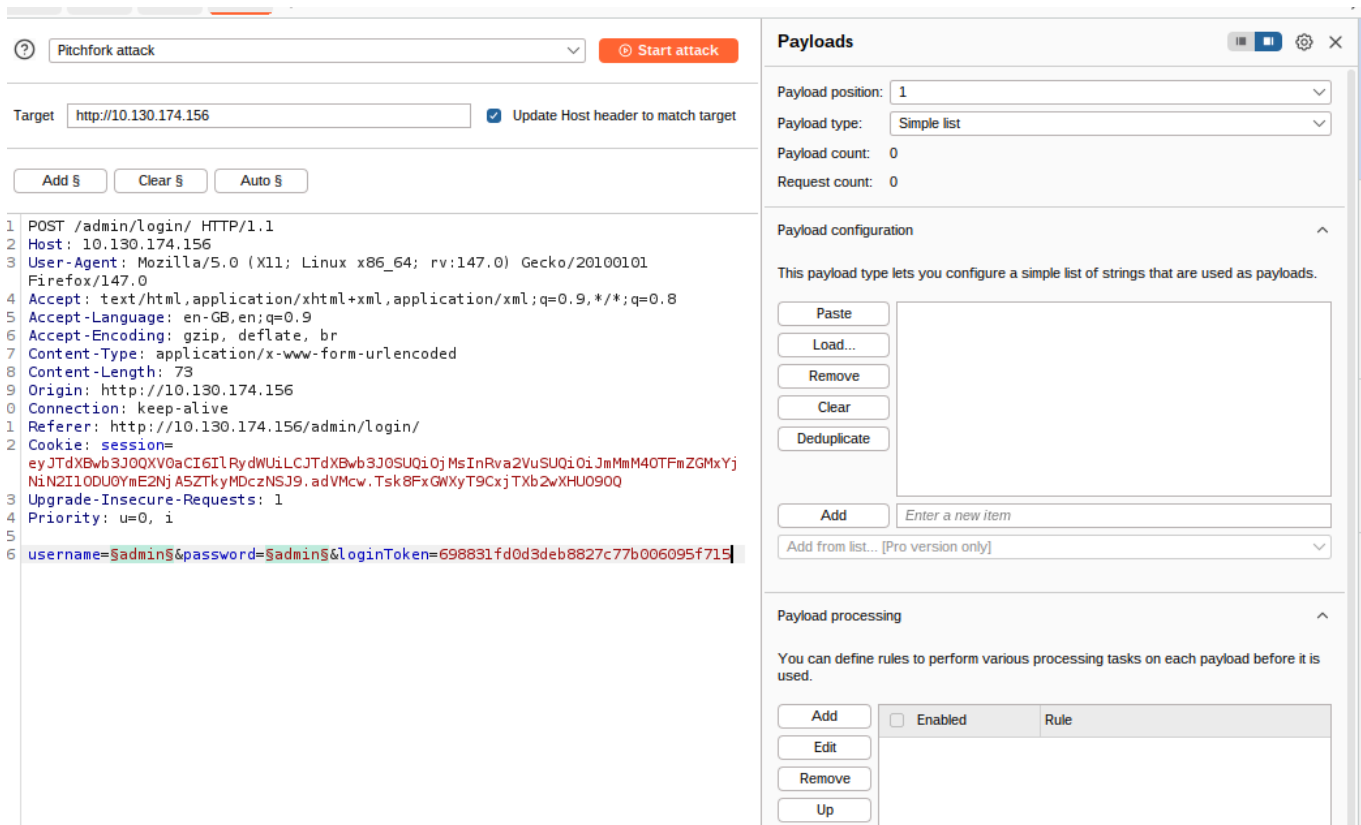
Extra Mile Challenge

this time, additional measures have been implemented to make brute-forcing more difficult.

```
HTTP/1.1 200 OK
Server: nginx/1.18.0 (Ubuntu)
Date: Fri, 20 Aug 2021 22:31:16 GMT
Content-Type: text/html; charset=utf-8
Connection: close
Set-Cookie: session=eyJ0b2t1bk1EIjoimzUyNTQ5ZjgxZDRhOTM5YjVlMTNlMjIzNmI0ZDlkOGEi-fQ.
Vary: Cookie
Front-End-Https: on
Content-Length: 3922
---
<form method="POST">
  <div class="form-floating mb-3">
    <input class="form-control" type="text" name=username placeholder="Usernam
    <label for="username">Username</label>
  </div>
  <div class="form-floating mb-3">
    <input class="form-control" type="password" name=password placeholder="Pas
    <label for="password">Password</label>
  </div>
  <input type="hidden" name="loginToken" value="84c6358bbf1bd8000b6b63ab1bd77c5e'
  <div class="d-grid"><button class="btn btn-warning btn-lg" type="submit">Login!
</form>
```

alongside the username and password fields, there is now a session cookie set, as well as a CSRF (**Cross-Site Request Forgery**) token in the form as a hidden field

Refreshing the page reveals that both the **session** cookie and the **loginToken** change with each request. This means that for every login attempt, we need to extract valid values for both the session cookie and the loginToken.



I entered the username and password wordlists like earlier into the payloads

With the username and password parameters handled, we now need to find a way to grab the ever-changing loginToken and session cookie

Macros allow us to perform the same set of actions repeatedly. In this case, we simply want to send a GET request to `/admin/login/`.

Process to set this up:

Fortunately, setting this up is a straightforward process.

- Switch over to the main "Settings" tab at the top-right of Burp.
- Click on the "Sessions" category.
- Scroll down to the bottom of the category to the "Macros" section and click the **Add** button.
- The menu that appears will show us our request history. If there isn't a GET request to `http://10.130.174.156/admin/login/` in the list already, navigate to this location in your browser, and you should see a suitable request appear in the list.
- With the request selected, click **OK**.
- Finally, give the macro a suitable name, then click **OK** again to finish the process.

Settings

Search

AllUserProject

Tools

Proxy

Intruder

Repeater

Sequencer

Burp's browser

Project

Sessions

Network

User interface

Suite

Extensions

Configuration library

Sessions

Use the sessions tracer to monitor or troubleshoot the behavior of your application.

Open sessions tracer

Cookie jar

Burp maintains a cookie jar that stores all of the cookies it receives from your application. You can use this to maintain valid sessions with applications that are being tested based on traffic from particular tools.

Use these settings to control which tools can update the cookie jar.

☒ Proxy

☐ Scanner

☐ Repeater

☐ Intruder

☐ Sequencer

☐ Extensions

Open cookie jar

Macros

A macro is a sequence of one or more requests. You can use macros to simulate a user's actions, such as logging in to an application, obtaining anti-CSRF tokens, etc. Use these settings to control which tools can create and edit macros.

Add

Edit

Remove

Duplicate

Macro Recorder - Macro Editor

Macro Recorder

Select the items from the proxy history that you wish to include in the macro, and click "Record". When you run the macro in your browser you will need to ensure that proxy interception is turned off.

Filter settings: Hiding CSS, image and general binary content

#	Host	Method	URL
1	http://10.130.174.156	POST	/support/login/
45	http://10.130.174.156	GET	/support/ticket/6
52	http://10.130.174.156	POST	/admin/login/
2	http://detectportal.firefox.com	GET	/canonical.html
3	http://detectportal.firefox.com	GET	/canonical.html
4	http://detectportal.firefox.com	GET	/canonical.html
5	http://detectportal.firefox.com	GET	/canonical.html
6	http://detectportal.firefox.com	GET	/canonical.html
7	http://detectportal.firefox.com	GET	/canonical.html
8	http://detectportal.firefox.com	GET	/canonical.html
9	http://detectportal.firefox.com	GET	/canonical.html
10	http://detectportal.firefox.com	GET	/canonical.html
11	http://detectportal.firefox.com	GET	/canonical.html

Request

PrettyRawHex

1 POST /admin/login/ HTTP/1.1

2 Host: 10.130.174.156

3 User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:147.0) Gecko/20100101 Firefox/147.0

4 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8

5 Accept-Language: en-GB,en;q=0.9

6 Accept-Encoding: gzip, deflate, br

7 Content-Type: application/x-www-form-urlencoded

8 Content-Length: 73

0 highlights

5. Now that we have a macro defined, we need to set Session Handling rules that define how the macro should be used.

- Still in the "Sessions" category of the main settings, scroll up to the "Session Handling Rules" section and choose to **Add** a new rule.
- A new window will pop up with two tabs in it: "Details" and "Scope". We are in the Details tab by default.

 Window showing the details and scope

- Fill in an appropriate description, then switch to the Scope tab.
- In the "Tools Scope" section, deselect every checkbox other than Intruder – we do not need this rule to apply anywhere else.
- In the "URL Scope" section, choose "Use suite scope"; this will set the macro to only operate on sites that have been added to the global scope (as was discussed in [Burp Basics](#)). If you have not set a global scope, keep the "Use custom scope" option as default and add `http://10.130.174.156/` to the scope in this section.

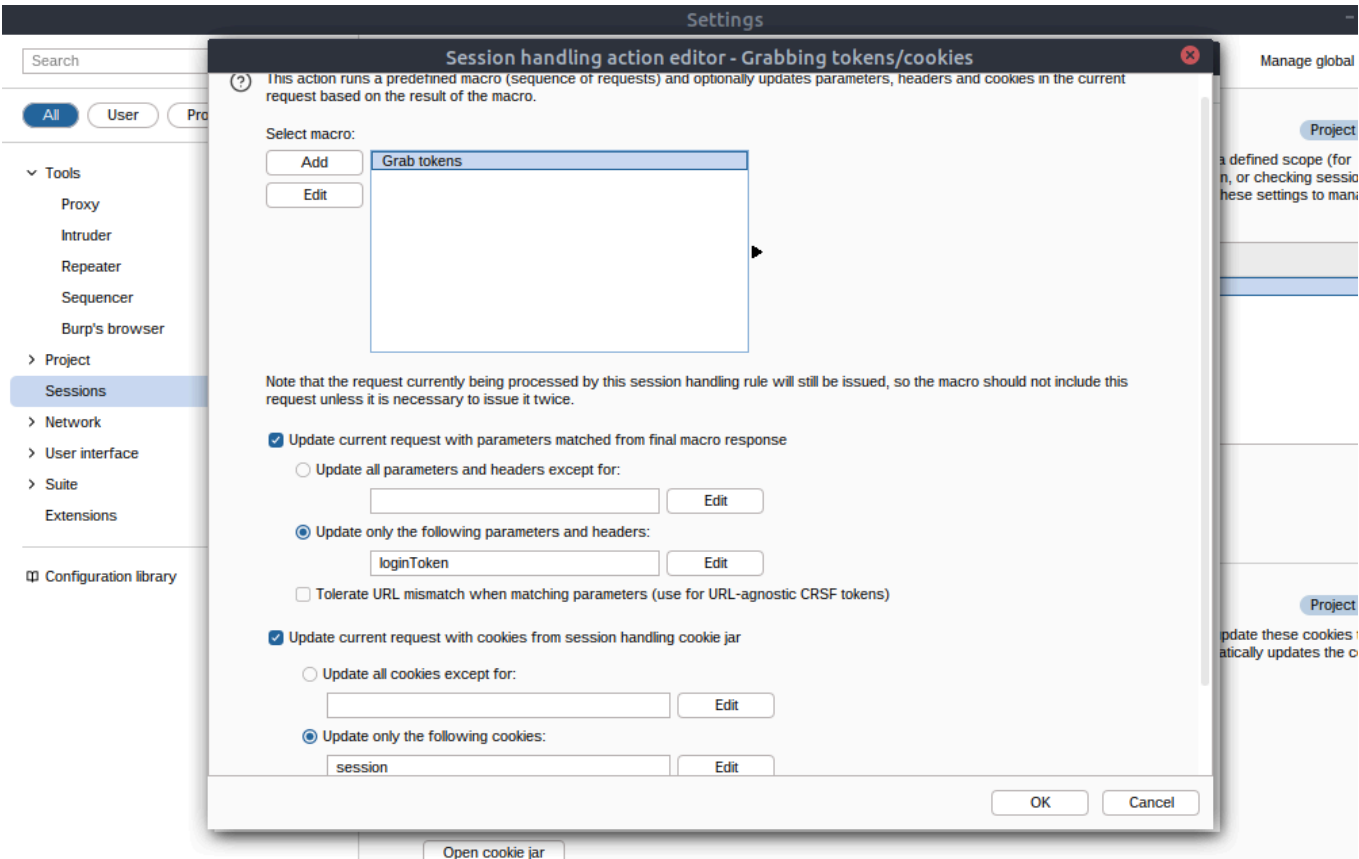
6. Now we need to switch back over to the Details tab and look at the "Rule Actions" section.

- Click the **Add** button – this will cause a dropdown menu to appear with a list of actions we can add.
- Select "Run a Macro" from this list.
- In the new window that appears, select the macro we created earlier.

As it stands, this macro will now overwrite all of the parameters in our Intruder requests before we send them; this is great, as it means that we will get the loginTokens and session cookies added straight into our requests. That said, we should restrict which parameters and cookies are being updated before we start our attack:

- Select "Update only the following parameters and headers", then click the **Edit** button next to the input box below the radio button.
- In the "Enter a new item" text field, type "loginToken". Press **Add**, then **Close**.
- Select "Update only the following cookies", then click the relevant **Edit** button.
- Enter "session" in the "Enter a new item" text field. Press **Add**, then **Close**.
- Finally, press **OK** to confirm our action.

this is setting the macro and adding the parameters



I started the attack and like before the response by length helps identify the valid credentials

ResultsPositions

Intruder attack results filter: Showing all items

Request	Payload 1	Payload 2	Status code	Response r...	Error	Timeout	Length ^	Comment
35	o.bennett	bella1	302	317			632	
13	r.miles	29011985	302	5			727	
18	k.house	tigge	302	5			727	
23	s.lester	rfnthbyf	302	4			727	
32	v.delgado	angel	302	5			727	
36	b.buckner	elefant	302	5			727	
57	s.ford	sanders	302	4			727	
58	j.perry	Sailing<3	302	4			727	
59	b.hebert	benessere	302	4			727	
70	j.oliver	marianna	302	5			727	
74	i.stout	beemer	302	4			727	